

Recursividad

La recursividad es un concepto muy utilizado en programación y se refiere a la capacidad de una función de llamarse a sí misma. En Python, al igual que en muchos otros lenguajes de programación, podemos crear funciones recursivas para resolver problemas de una manera más elegante y concisa.

Una función recursiva comienza con un caso base, que es un caso simple que se puede resolver sin necesidad de una llamada recursiva. Luego, se define un caso recursivo, que es el caso en el que la función se llama a sí misma con argumentos modificados. Cada vez que la función se llama a sí misma, el problema se divide en un subproblema más pequeño, que se resuelve llamando a la función recursiva de nuevo. Este proceso continúa hasta que se alcanza el caso base, que es cuando se devuelve el resultado final.

La recursividad en Python puede ser muy útil para resolver problemas que se pueden descomponer en subproblemas más pequeños y similares. Un ejemplo clásico es el cálculo del factorial de un número entero. El factorial de un número n se define como la multiplicación de todos los números enteros positivos desde 1 hasta n . Podemos definir la función factorial recursivamente de la siguiente manera:

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n-1)
```

En este ejemplo, el caso base es cuando n es igual a 1, en cuyo caso se devuelve 1. En el caso recursivo, la función se llama a sí misma con el argumento $n-1$. Esto significa que el problema se divide en subproblemas más pequeños hasta que se llega al caso base, donde se devuelve el resultado final.

Es importante tener en cuenta que la recursividad puede tener un costo alto en términos de memoria y tiempo de ejecución. Si no se definen correctamente los casos base y recursivos, puede haber un riesgo de que la función entre en un ciclo infinito, lo

que puede hacer que el programa se bloquee o se agote la memoria disponible. Por lo tanto, es importante utilizar la recursividad de manera cuidadosa y adecuada.

Ejemplos:

Ejemplo1: Factorial recursivo

Escribe una función recursiva en Python que calcule el factorial de un número entero. El número debe ser ingresado por el usuario y el resultado debe ser mostrado en pantalla.

```
def factorial(n):  
    """  
    Esta función calcula el factorial de un número de forma recursiva.  
    """  
    # Caso base  
    if n == 1:  
        return 1  
    # Caso recursivo  
    else:  
        return n * factorial(n-1)  
  
# Ejemplo de uso  
resultado = factorial(5)  
print(resultado) # Debería imprimir 120
```

Este programa calcula el factorial de un número utilizando una función recursiva. En este programa, la función `factorial()` es recursiva. El caso base es cuando `n` es igual a 1, en ese caso, la función devuelve 1. En caso contrario, la función se llama a sí misma con el parámetro `n-1` y multiplica el resultado por `n`.

Ejemplo2: Fibonacci recursivo

Escribe una función recursiva en Python que genere la secuencia de Fibonacci hasta un número entero ingresado por el usuario. El número debe ser ingresado por el usuario y la secuencia debe ser mostrada en pantalla.

```
def fibonacci(n):  
    """  
    Esta función genera la secuencia de Fibonacci de forma recursiva.  
    """  
    # Casos base
```

```

if n == 0:
    return 0
elif n == 1:
    return 1
# Caso recursivo
else:
    return fibonacci(n-1) + fibonacci(n-2)

# Ejemplo de uso
resultado = fibonacci(10)
print(resultado) # Debería imprimir 55

```

Este programa genera la secuencia de Fibonacci utilizando una función recursiva.

En este programa, la función `fibonacci()` es recursiva. Los casos base son cuando `n` es igual a 0 o 1, en esos casos, la función devuelve 0 o 1, respectivamente. En caso contrario, la función se llama a sí misma con los parámetros `n-1` y `n-2` y devuelve la suma de los dos resultados.

Ejemplo3: Búsqueda binaria recursiva

Escribe una función recursiva en Python que implemente la búsqueda binaria en una lista de números enteros. La función debe recibir como parámetros la lista de números y el número a buscar, y debe devolver el índice del número en la lista o -1 si el número no se encuentra en la lista. La lista de números debe ser ingresada por el usuario y el número a buscar debe ser generado aleatoriamente. El resultado de la búsqueda debe ser mostrado en pantalla.

```

def busqueda_binaria(lista, valor, inicio=0, fin=None):
    """
    Esta función busca un valor en una lista utilizando la búsqueda binaria de
    forma recursiva.
    """
    # Si no se especifica el valor de fin, se toma el valor máximo
    if fin == None:
        fin = len(lista) - 1

    # Caso base: el valor no está en la lista
    if inicio > fin:
        return -1

    # Calculamos el índice medio
    medio = (inicio + fin) // 2

    # Caso base: el valor está en el índice medio
    if lista[medio] == valor:
        return medio

```

```
# Caso recursivo: buscamos en la mitad izquierda o derecha de la lista
elif lista[medio] > valor:
    return busqueda_binaria(lista, valor, inicio, medio-1)
else:
    return busqueda_binaria(lista, valor, medio+1, fin)

# Ejemplo de uso
lista = [1, 3, 5, 7, 9]
indice = busqueda_binaria(lista, 7)
print(indice) # Debería imprimir 3
```

Este programa busca un elemento en una lista utilizando una función recursiva que implementa la búsqueda binaria.

En este programa, la función `busqueda_binaria()` es recursiva. Los casos base son cuando el valor no está en la lista (el índice de inicio es mayor que el índice de fin) o cuando

Ejercicios

Ejercicio 1: Suma recursiva

Escribe una función recursiva en Python que calcule la suma de los primeros n números enteros. El valor de n debe ser ingresado por el usuario y el resultado debe ser mostrado en pantalla. Utiliza la recursividad para calcular la suma.

Ejercicio 2: Potencia recursiva

Escribe una función recursiva en Python que calcule la potencia de un número entero. El número base y el exponente deben ser ingresados por el usuario y el resultado debe ser mostrado en pantalla. Utiliza la recursividad para calcular la potencia.

Entrega en parejas